MAZE · ЗАЩИТА ПРОЕКТА

Maze.

Лабиринты, пещеры, BFS и Q-learning: как устроен проект, где лежит каждая часть и что говорить на защите.

KOTLIN 1.9.22 · COMPOSE DESKTOP · KTOR 2.3.13 · VUE 3 · Q-LEARNING · 81 ТЕСТ

6	4	81	100%
частей ТЗ	алгоритма	тест	покрытие алгоритмов

Открывайте в браузере: стрелки ← →, пробел, **T** — оглавление, **F** — полный экран.



ОБЗОР

Карта ТЗ: что защищаем

Шесть частей проекта: от модели лабиринта до веб-интерфейса. Всё, что отмечено «готово», лежит в src/maze/app/src.

PART 1 — ГОТОВО

Лабиринт

Файл, модель, загрузка, отрисовка 500×500, стены 2 px.

`1..50 · 0/1 · 2 px`

`Maze.kt · MazeFileRepository.kt · MazeCanvas.kt`

PART 2 — ГОТОВО

Генерация

Идеальный лабиринт алгоритмом Эллера, без петель и изолированных зон.

`sets · merge · openings`

`EllerMazeGenerator.kt`

PART 3 — ГОТОВО

Решение

BFS, кратчайший маршрут, линия 2 px через центры клеток.

`queue · previous · buildPath`

`MazeSolver.kt · MazeCanvas.kt`

PART 4 — ГОТОВО

Пещеры

Клеточный автомат, шаг/авто, загрузка и сохранение.

`8 соседей · birth/death 0..7`

`Cave.kt · CaveGenerator.kt · CaveScreen.kt`

PART 5 — ГОТОВО

Q-learning

Агент учится проходить лабиринт из любой точки до фиксированного финиша.

`Q(s,a) · ε-greedy · эпизоды`

`QLearningAgent.kt · AgentScreen.kt`

PART 6 — ГОТОВО

Web

Vue-страница + Ktor API; генерация, файлы и решение на том же Kotlin-ядре.

`GET/POST · JSON вручную`

`WebServer.kt · web/index.html`

Ключевая идея: UI и веб — только тонкие оболочки. Вся логика в доменном слое, алгоритмы ничего не знают про Compose, Ktor и файлы.

1. Эллер 50×50
2. BFS-маршрут
3. Пещера: шаг/авто
4. Агент 10×10
5. Web: /api/solve

ИНФРАСТРУКТУРА

Стек и как он подключён

Gradle-сборка, Compose Desktop, Ktor, Vue 3 и тестовый контур. В зависимостях нет RL-библиотек, JSON-библиотек и DI-фреймворков.

K

Kotlin + JVM 21

`1.9.22 / toolchain 21`

Язык всего проекта. Версия Java задана в buildSrc convention-плагине.

`toolchain(21) · buildSrc convention`

`buildSrc/.../kotlin-jvm.gradle.kts`

▣

Compose Desktop

`1.6.0`

Окно, вкладки, Canvas 500×500, кнопки и поля ввода.

`compose.desktop.currentOs`

`app/build.gradle.kts · ui/`

⇄

Ktor + Netty

`2.3.13`

Встроенный HTTP-сервер: страница, API, JSON руками.

`ktorServerCore + ktorServerNetty`

`web/WebServer.kt · Launcher.kt`

V

Vue 3

`global prod`

SPA без CDN: vue.global.prod.js лежит в ресурсах.

`GET /vue.global.prod.js · без npm`

`resources/web/`

✓

kotlin.test + JUnit 5

`BOM 5.10.2`

81 тест, JUnit Platform, отчёт о покрытии JaCoCo.

`kotlin("test") + JaCoCo от buildSrc`

`app/src/test/kotlin`

⚙

Gradle 9.3.0

`wrapper + buildSrc`

Мультипроект, convention-плагин, caching, toolchain.

`gradlew + libs.versions.toml`

`gradlew · settings.gradle.kts`

≡

ktlint + Dokka

`14.2.0 / 1.9.20`

Стиль Kotlin и HTML-документация из KDoc (make ktlint, make dvi).

`ktlint 14.2.0 · Dokka 1.9.20`

`.editorconfig · build.gradle.kts`

M

Makefile

`GNU targets`

all, install, uninstall, clean, dvi, dist, tests + run/ktlint.

`make run/test/build/dvi/dist`

`Makefile`

ЗАПУСК
`make run` или `./gradlew run` → окно + `http://localhost:9090`

ПРОВЕРКА
`make tests` · `./gradlew ktlintCheck` · `./gradlew jacocoTestReport`

СБОРКА
`make build` · `make install` · `make dist`

buildSrc
toolchain 21 · JUnit · JaCoCo
→
app
Compose + Ktor + resources
→
GameService
фасад над алгоритмами
→
domain
Maze · Cave · Q-table
→
UI / Web
две тонкие оболочки

НАВИГАЦИЯ

Дерево проекта: где что искать

Слева — реальная структура src/maze. Справа — что лежит в каждом слое и куда идти за ответом.

src/maze — Kotlin

```
app
  src/main/kotlin
    domain
      algorithm
        EllerMazeGenerator.kt
        MazeSolver.kt
        CaveGenerator.kt
        QLearningAgent.kt
      model
        Maze.kt
        Cave.kt
      service
        GameInterface.kt
        GameService.kt
    database
      repository
        MazeFileRepository.kt
        CaveFileRepository.kt
      mapper
        MazeMapper.kt
      model
        MazeFileModel.kt
    ui
      MazeScreen.kt
      CaveScreen.kt
      AgentScreen.kt
      MazeCanvas.kt
      Common.kt
      mapper
        MazeUiMapper.kt
    di
      AppContainer.kt
    web
      WebServer.kt
    App.kt
    Launcher.kt
  src/test/kotlin                    11
  src/main/resources                  3
  build.gradle.kts
Makefile
gradle/libs.versions.toml
buildSrc/src/main/kotlin/kotlin-jvm.gradle.kts
```

Домен и алгоритмы

- `domain/algorithm` — 4 алгоритма: Эллер, BFS, пещера, Q-learning.
- `domain/model` — Maze, Cave, Position: только данные и правила.
- `domain/service` — GameInterface/GameService: фасад для UI.

Данные и UI

- `database` — парсинг/сохранение файлов и MazeMapper.
- `ui` — три вкладки, Canvas и маппер координат.
- `web` — Ktor-роуты; страница лежит в resources/web.

Тесты

- `app/src/test/kotlin` — 11 классов, 81 тест.
- Алгоритмы покрыты на 100% по строкам и ветвям; UI проверяется вручную.

Цвета в дереве

- algorithm — Эллер, BFS, пещера, агент
- model — Maze, Cave, Position
- database — repository, mapper
- ui — экраны, Canvas, маппер

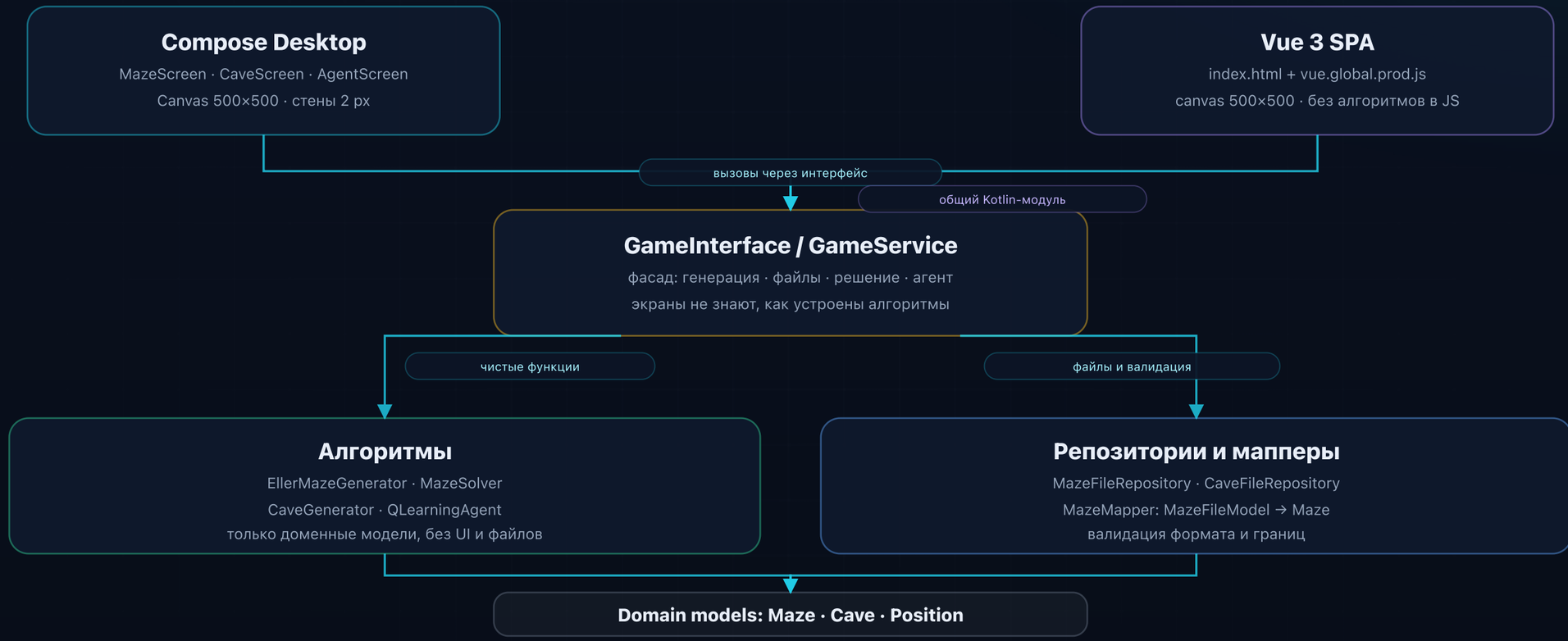
Куда смотреть на защите

QLearningAgent.kt	слайд 19
EllerMazeGenerator.kt	слайд 11
MazeSolver.kt	слайд 13
WebServer.kt	слайд 25

КАК СВЯЗАНО

Архитектура: слои и изоляция

Одна и та же доменная логика используется и в desktop, и в web. UI знает только интерфейс GameInterface.



Сборка зависимостей: AppContainer вручную создаёт репозитории, GameService и экраны; web получает тот же сервис. DI-библиотек и глобального состояния нет.

UI не знает алгоритмов
Экраны вызывают GameService, а не EllerMazeGenerator или QLearningAgent напрямую.

Web использует то же ядро
Ktor принимает текст лабиринта, вызывает GameService и отдаёт JSON.

Данные изолированы
MazeFileModel ≠ Maze; UiPoint/UiLine ≠ Position. Между ними — мапперы.

СКВОЗНОЙ ПУТЬ

Поток данных: от файла до экрана

Один и тот же маршрут проходят лабиринт, пещера и веб-запрос: разбор → валидация → домен → алгоритм → отрисовка.

ВХОД .txt по ТЗ · HTTP body · поля UI → **ВЫХОД** Maze/Cave · path · JSON · Canvas



сырой текст | проверки | изоляция формата | единая модель | чистая логика | только отрисовка

Ошибка на любом шаге превращается в понятное сообщение пользователю, а не в падение приложения

Валидация на входе
Размер 1..50, число строк/столбцов, только 0/1, сплошная внешняя граница.

Домен ничего не знает о файлах
Maze — это матрицы стен и canGo; формат живёт только в repository/mapper.

Отрисовка только по модели
Canvas не парсит текст: он получает Maze/Cave/Position и рисует.

Размер вне диапазона
`rows = 51`
«Размеры лабиринта должны быть от 1 до 50»

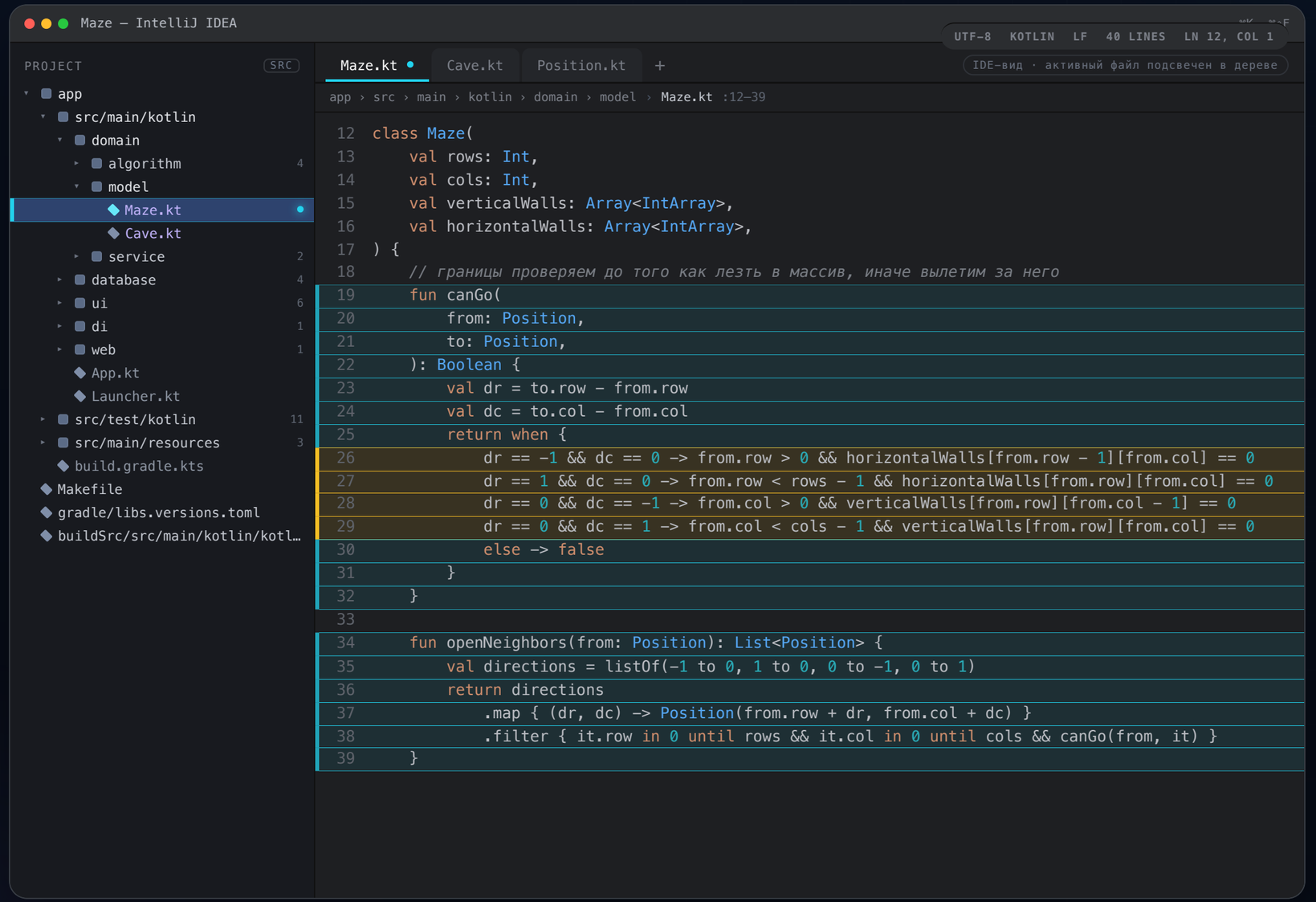
Значение не 0/1
`2`
«Ожидалось 0 или 1» — файл не попадёт в домен.

Координата вне поля
`startRow = 99`
HTTP 400 + JSON с сообщением, без stack trace.

Один **parse → Maze** используют экран, тесты и HTTP: три входа — один домен.

Part 1: модель лабиринта

Maze — это размеры, две матрицы стен и два правила: canGo и openNeighbors. Матрицы хранят правую и нижнюю стенку клетки.



Как читать матрицы

- `verticalWalls[r][c] == 1` — стена справа от клетки (r,c).
- `horizontalWalls[r][c]` — стена снизу.
- Последний столбец и последняя строка — внешняя граница; верхнюю и левую границу дорисовывает UI.

canGo: единственная точка проверки перехода

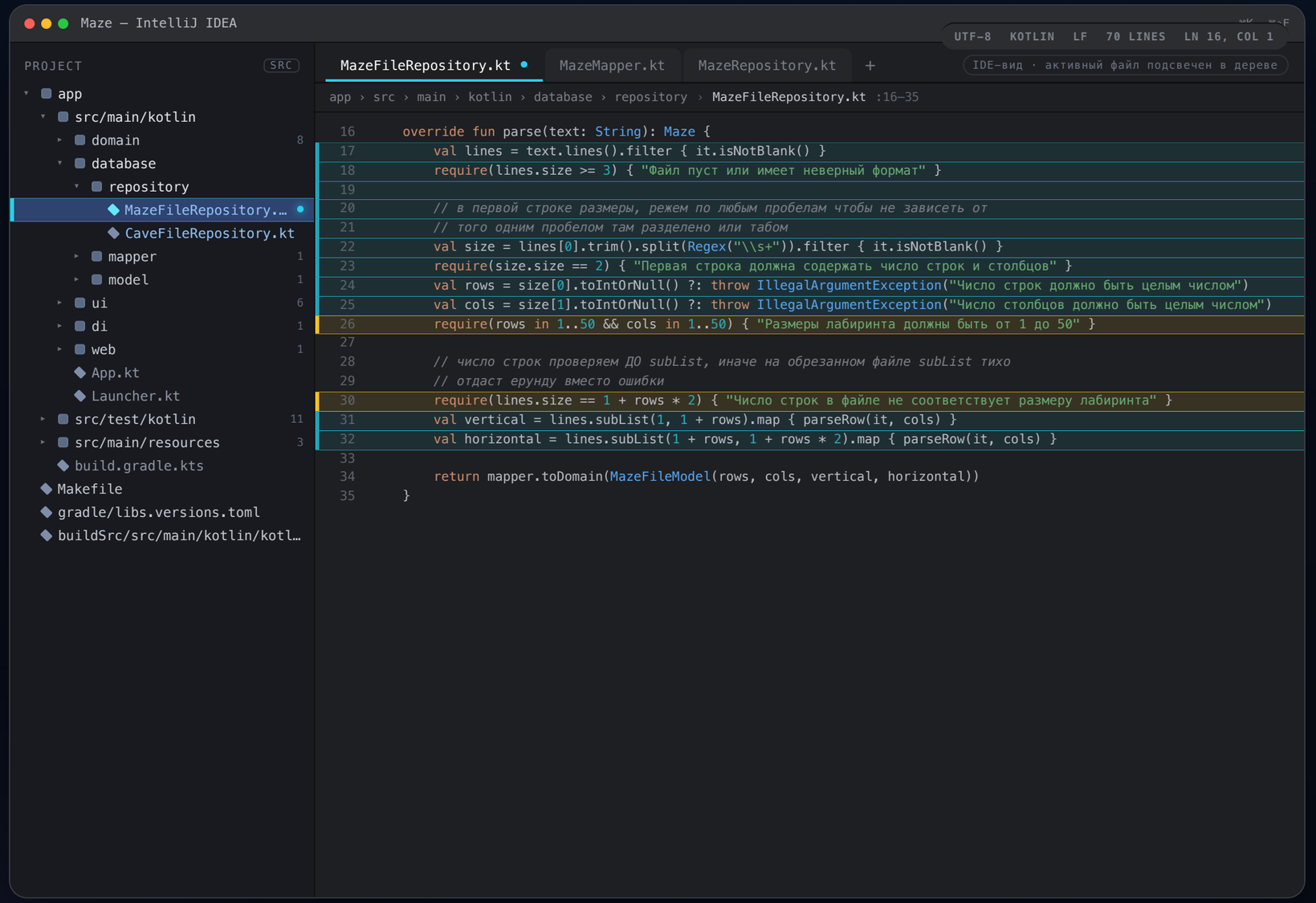
- `dr/dc` определяют направление; для каждого — своя матрица и проверка границы.
- Если переход не соседний или упирается в стену — false.
- `openNeighbors` использует canGo; на нём стоят BFS и тесты.

Как это связано дальше

- BFS и агент вызывают canGo, поэтому не знают о файлах и UI.
- MazeUiMapper переводит Position в UiPoint и линии для Canvas.
- MazeModelTest: 7 проверок границ и открытых соседей.

Part 1: загрузка, парсинг, валидация

Пользователь выбирает файл; repository читает текст и проверяет формат до того, как он попадёт в домен.



Проверки в parse()

- Первая строка — ровно два целых числа.
- Размеры `1..50`; иначе — IllegalArgumentException.
- Строк в файле ровно `1 + rows * 2`: две матрицы.
- В каждой строке ровно cols значений, каждое 0 или 1.
- Пустые строки фильтруются, поэтому файл без разделителя тоже читается.

Проверки в MazeMapper.toDomain

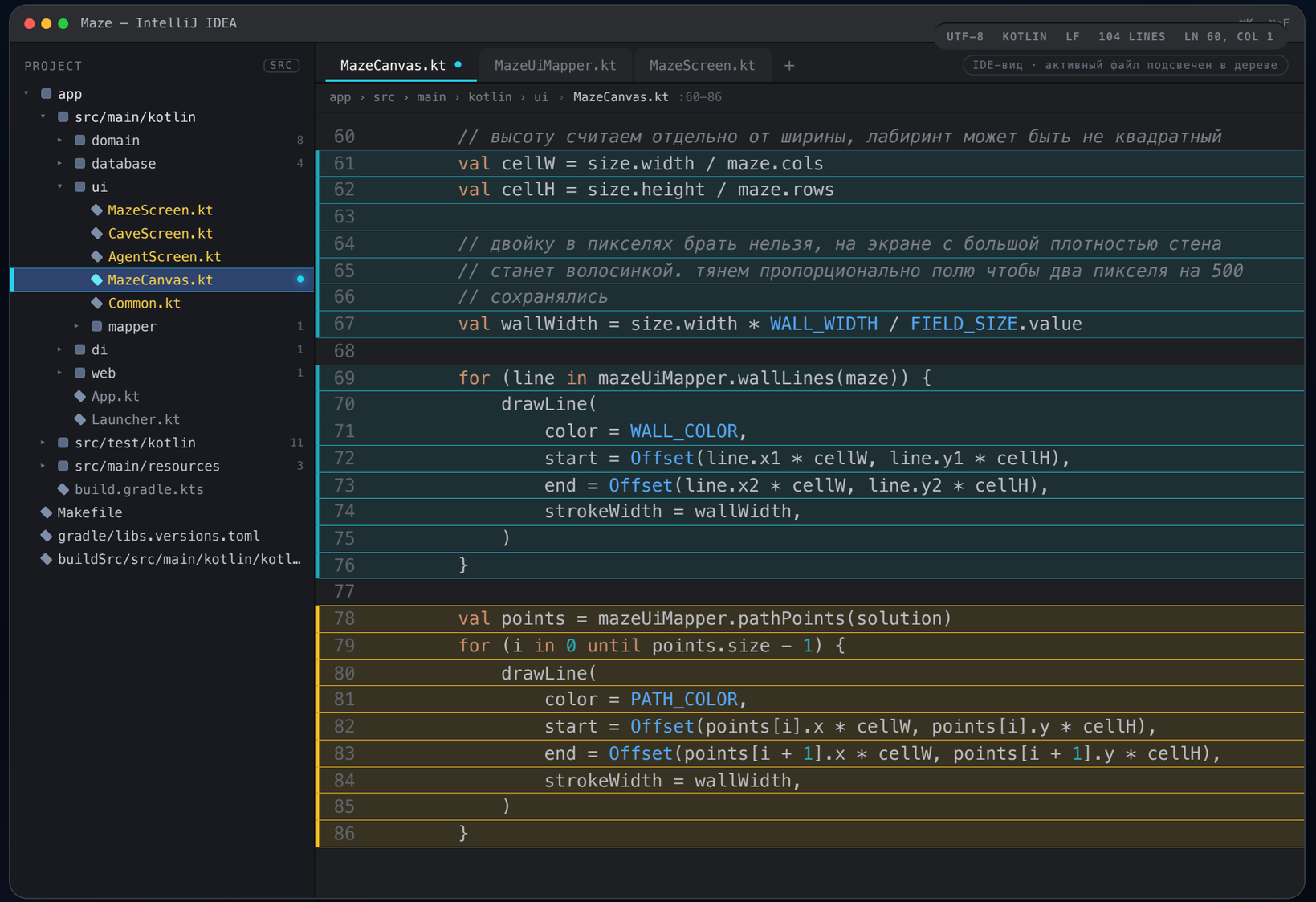
- Размеры и размеры матриц совпадают.
- Правая граница: `verticalWalls[r][cols-1] == 1`.
- Нижняя граница: `horizontalWalls[rows-1][c] == 1`.
- Дырявая внешняя граница ломает canGo — лучше отловить сразу.

Контракт формата

- 1 строка-размер + rows×2 строк матриц, cols чисел в каждой.
- Битый файл не доходит до Maze — исключение ловит UI или HTTP-слой.
- CaveFileRepository повторяет те же проверки для пещеры.

Part 1: отрисовка 500×500 и стены 2 px

MazeCanvas переводит клетки в пиксели: клетка = ширина/cols, стена = 2/500 ширины поля. Маршрут рисуется через центры.



Геометрия

- `cellW = size.width / cols`, `cellH = size.height / rows` — лабиринт занимает всё поле.
- `wallWidth = size.width * 2 / 500` — 2 пикселя при поле 500.
- Поле фиксировано: 500.dp; окно может быть больше, Canvas не растягивается.

Что рисуется

- MazeUiMapper.wallLines добавляет верхнюю и левую границу, затем все правые и нижние стены.
- Маршрут — красная линия, стены — тёмно-серые, поле — белое: цвет отличается.
- `pathPoints` сдвигает на +0.5 клетки, поэтому линия идёт через центры.

Где это в UI

- `MazeScreen` — вкладка лабиринта; `AgentScreen` переиспользует тот же Canvas.
- Клик по полю пересчитывается из пикселей в (row, col).

Part 2: алгоритм Эллера — идея

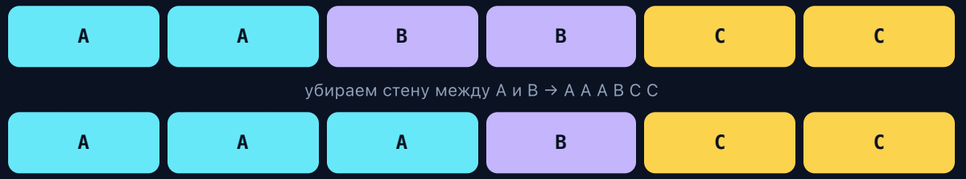
Эллер строит идеальный лабиринт построчно: клетки объединяются в множества, а стены убираются только между разными множествами.

01 **Множества**
Каждая клетка начинает в своём множестве. Одинаковый номер = клетки уже соединены.

02 **Правые стены**
Идём по строке: если соседи из разных множеств, случайно убираем стену и сливаем множества. Между своими — нельзя, иначе появится петля.

03 **Проходы вниз**
Случайно открываем нижние стены, но проверяем: у каждого множества должен остаться хотя бы один проход вниз.

04 **Новая строка**
Клетки без прохода вниз получают новые номера множеств. В последней строке просто сливаем все множества вправо.



Почему идеальный
Ровно N–1 проход, нет циклов (стены рвём только между разными множествами), все клетки достижимы (каждое множество получает выход вниз, последняя строка всё сливает).

N–1 проходов ровно

1 связная компонента

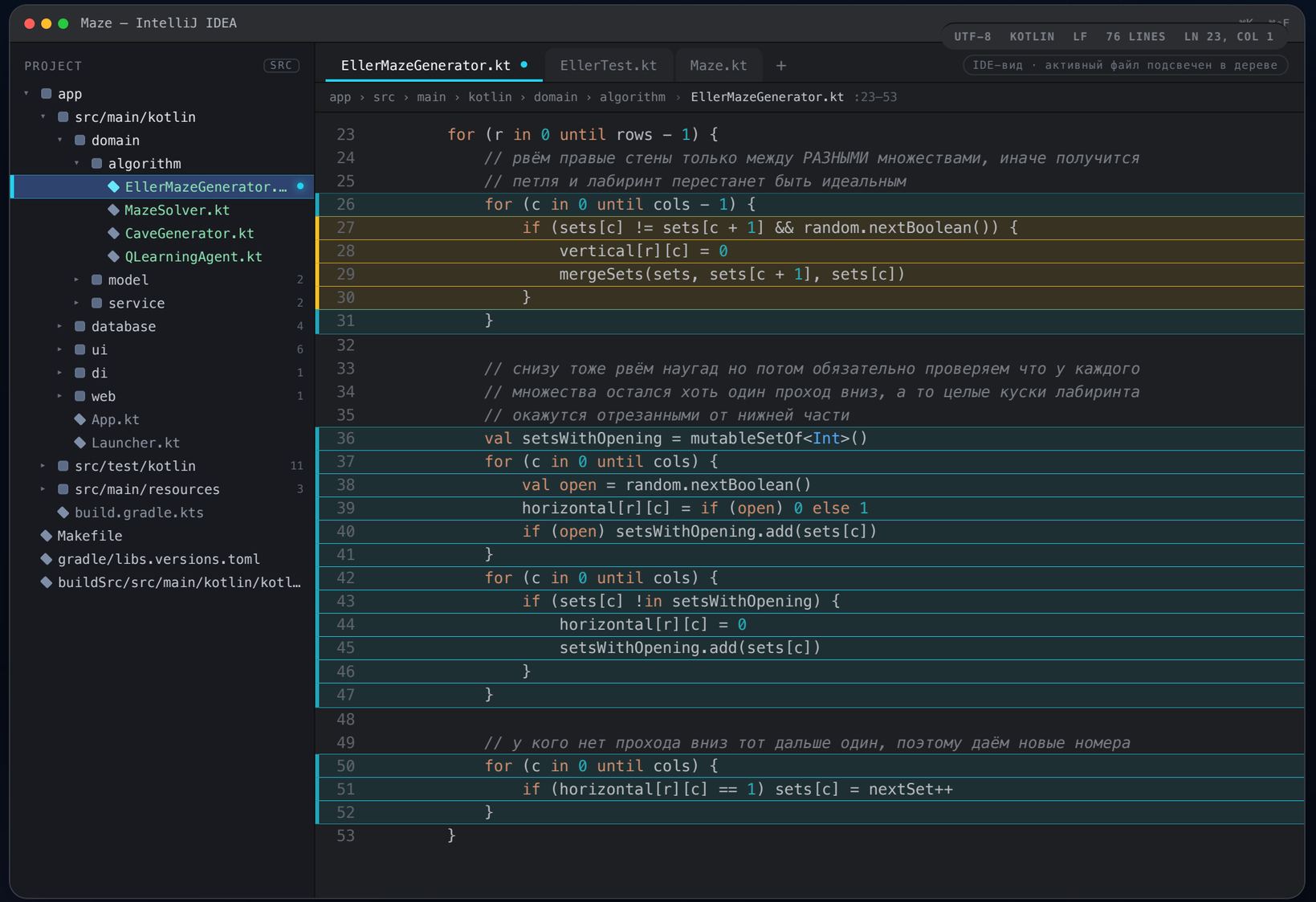
0 циклов

O(N) доп. память под sets

Для 1×1 алгоритм тоже работает: цикл по строкам не выполняется, возвращается одна клетка.

Part 2: код Эллера

Главный цикл: слияние множеств справа, гарантированные проходы вниз и новые номера для клеток, которые не продолжаются.



Правые стены

- Условие `sets[c] != sets[c+1]` — обязательное: иначе получится петля.
- `mergeSets` перекрашивает все клетки множества from в to.

Нижние проходы

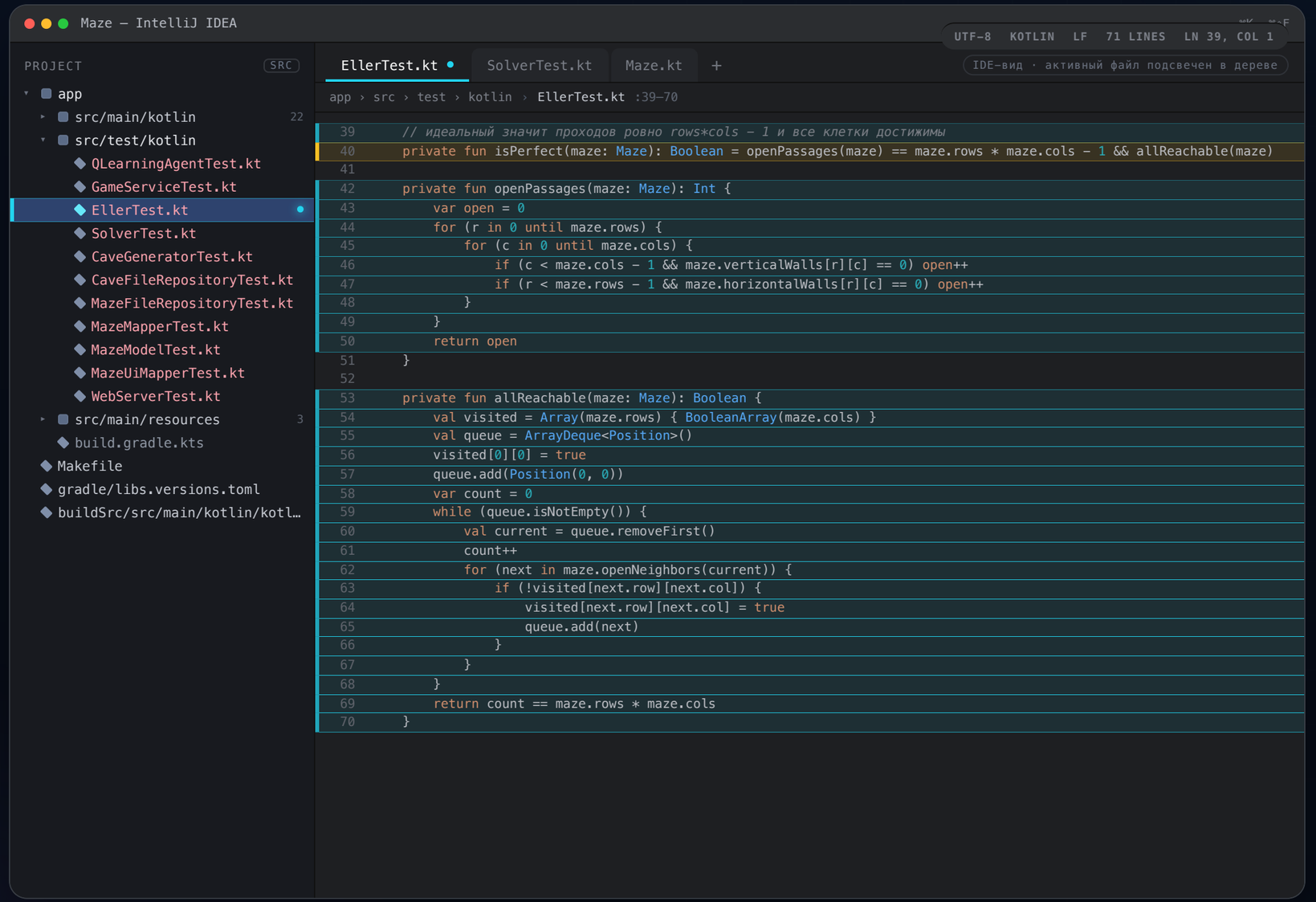
- `setsWithOpening` запоминает, у кого уже есть выход вниз.
- Второй проход по строке принудительно открывает стену у множеств без выхода.
- Это страховка от отрезанных областей.

Новая строка и последняя

- Клетки с закрытой нижней стеной получают новый set id.
- В последней строке (строки 55–64) убираем все правые стены между разными множествами — остаётся один связный лабиринт.

Part 2: почему лабиринт идеальный + тесты

Тест проверяет два свойства: проходов ровно N−1 и из左上 угла достижимы все клетки. Это и есть определение идеального лабиринта.



Определение в коде

- `openPassages == rows*cols – 1` — дерево, а не граф с циклами.
- `allReachable` — BFS из (0,0) должен посетить все клетки.
- Оба условия вместе = идеальный лабиринт.

Что ещё проверяется

- Все размеры от 1×1 до 10×10.
- Размеры 0 и 51 отклоняются.
- Покрытие EllerMazeGenerator — 100% строк и ветвей.

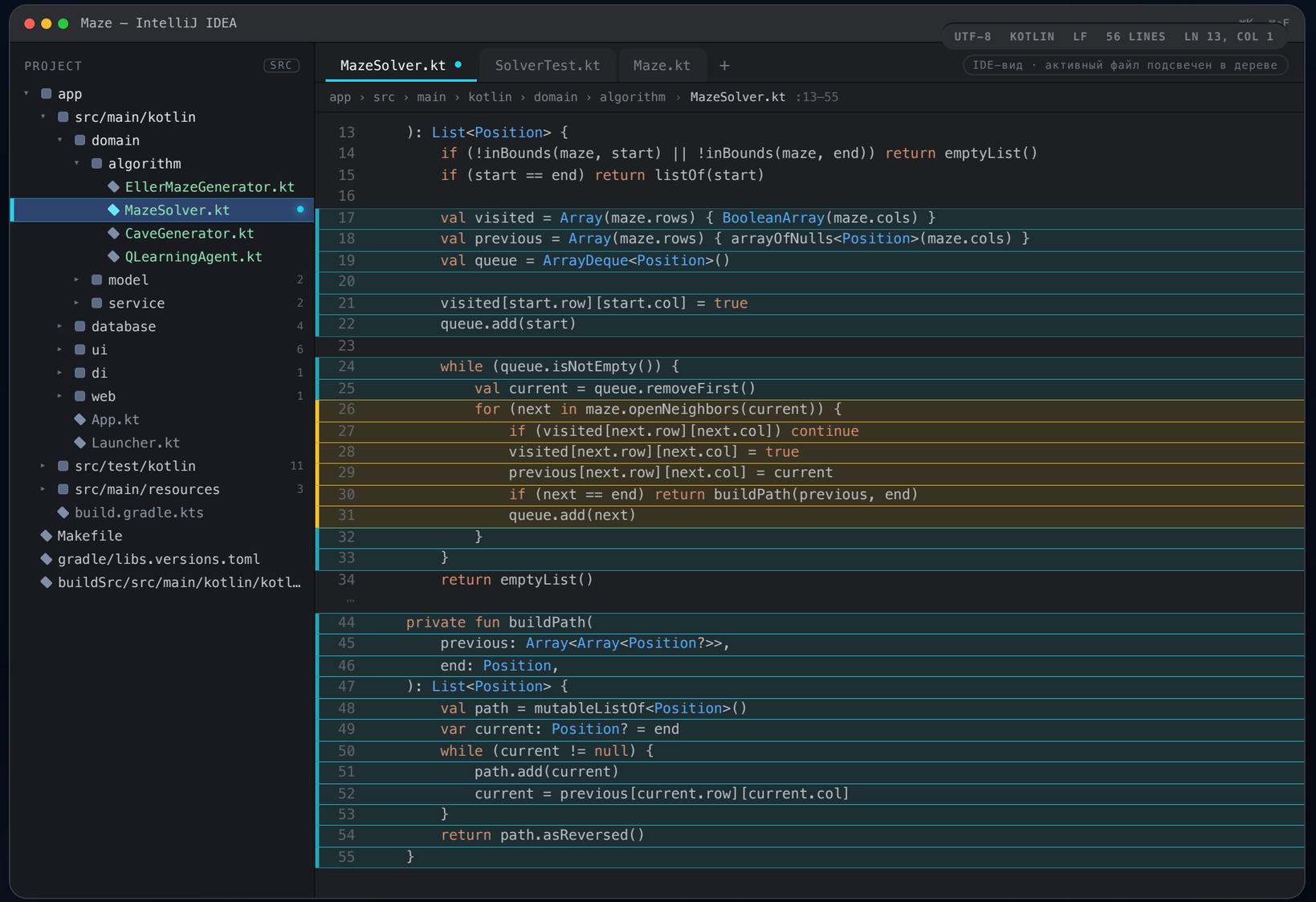
Почему этого достаточно

- N–1 проход + связность исключают и петли, и изолированные участки.
- Для 50×50 те же свойства выполняются по построению; граничный размер проверяется отдельно в других тестах.

ELLERTEST.KT:13–21, 39–70 · ПОКРЫТИЕ 100%

Part 3: BFS-решатель

Очередь + visited + previous. BFS гарантирует кратчайший путь по числу клеток; путь собирается назад от финиша.



Почему BFS

- Все переходы стоят одинаково — один шаг. BFS первым доходит до финиша по кратчайшему числу клеток.
- `visited` не даёт повторно положить клетку в очередь.

Восстановление пути

- `previous[r][c]` хранит, откуда пришли.
- Идём от финиша назад, пока не упрёмся в null (у старта предка нет), затем разворачиваем.

Границы

- Старт или финиш вне поля — emptyList.
- Старт = финиш — список из одной клетки.
- Нет пути — emptyList, UI показывает «Путь не найден».

Part 3: маршрут на экране

Маршрут — красная линия толщиной 2 px через центры клеток. Цвет отличается от стен и поля, как требует ТЗ.

Maze – IntelliJ IDEA

UTF-8 KOTLIN LF 104 LINES LN 69, COL 1

PROJECT SRC

- app
  - src/main/kotlin
    - domain 8
    - database 4
    - ui
      - MazeScreen.kt
      - CaveScreen.kt
      - AgentScreen.kt
      - MazeCanvas.kt
      - Common.kt
    - mapper 1
    - di 1
    - web 1
    - App.kt
    - Launcher.kt
  - src/test/kotlin 11
  - src/main/resources 3
  - build.gradle.kts
- Makefile
- gradle/libs.versions.toml
- buildSrc/src/main/kotlin/kotl…

MazeCanvas.kt • | MazeUiMapper.kt | SolverTest.kt | +

IDE-вид · активный файл подсвечен в дереве

app › src › main › kotlin › ui › MazeCanvas.kt :69–86

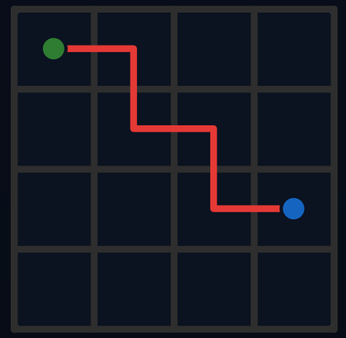
```
69          for (line in mazeUiMapper.wallLines(maze)) {
70              drawLine(
71                  color = WALL_COLOR,
72                  start = Offset(line.x1 * cellW, line.y1 * cellH),
73                  end = Offset(line.x2 * cellW, line.y2 * cellH),
74                  strokeWidth = wallWidth,
75              )
76          }
77
78          val points = mazeUiMapper.pathPoints(solution)
79          for (i in 0 until points.size - 1) {
80              drawLine(
81                  color = PATH_COLOR,
82                  start = Offset(points[i].x * cellW, points[i].y * cellH),
83                  end = Offset(points[i + 1].x * cellW, points[i + 1].y * cellH),
84                  strokeWidth = wallWidth,
85              )
86          }
```

Точки маршрута

- `MazeUiMapper.pathPoints` : (col + 0.5, row + 0.5) — ровно центр клетки.
- Линия рисуется между соседними точками; толщина та же, что у стен, — 2 px.
- Цвета: стены #2E2E2E, поле белое, маршрут #E53935.

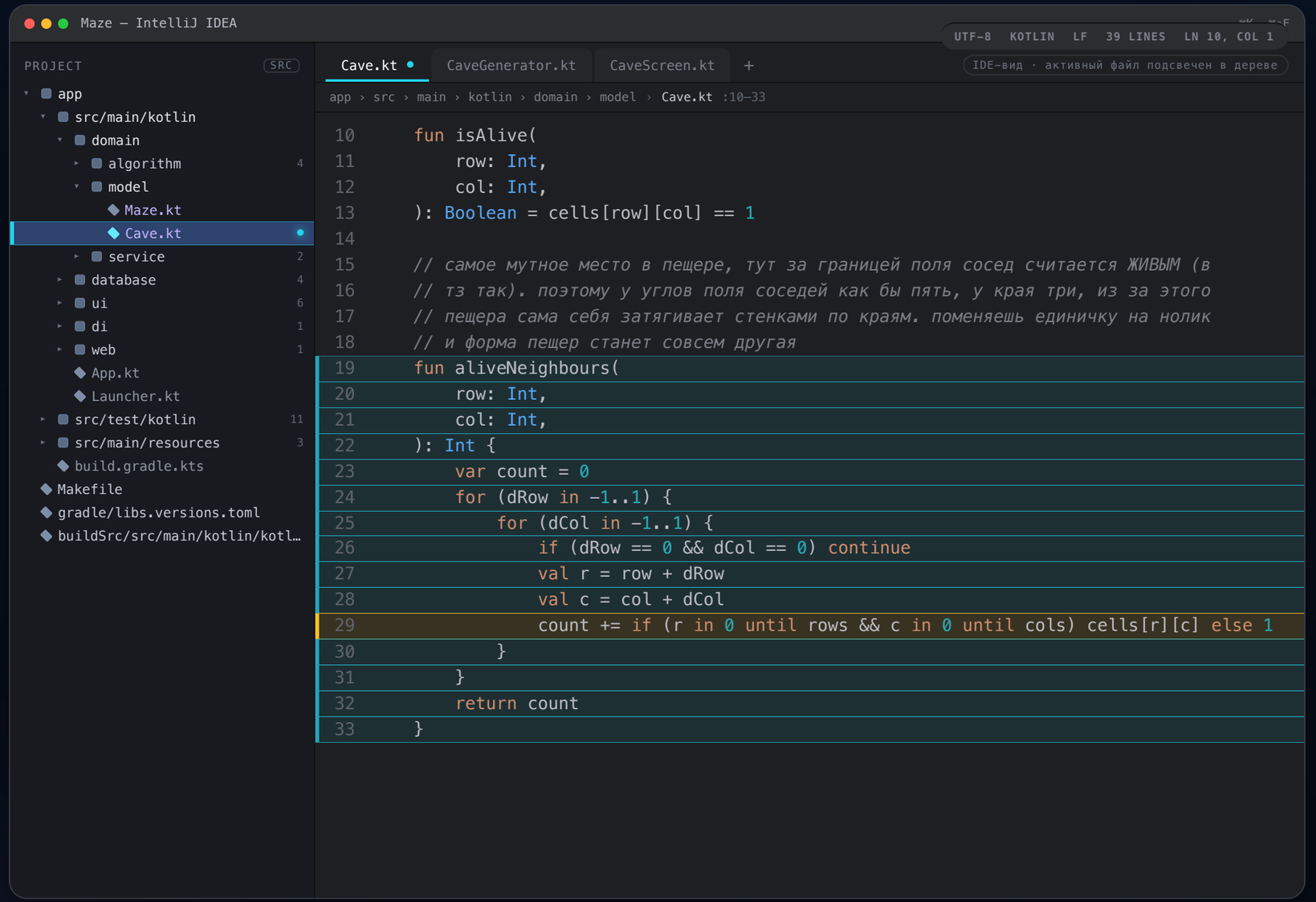
Что проверяют тесты

- Известный 2×2: конкретный путь.
- Из середины выбирается короткий из двух маршрутов.
- Все размеры 1..10, старт/финиш, выход за границы, отсутствие пути.



Part 4: пещера — модель и соседи

Пещера — матрица 0/1. Живая клетка — стена, мёртвая — свободно. Клетки за границей считаются живыми: это ключ к поведению автомата.



Кто есть кто

- `1` — живая клетка (стена), `0` — мёртвая (свободно).
- `aliveNeighbours` считает 8 соседей, центр пропускается.
- За границей `count += 1` — по ТЗ внешние клетки живые.

Следствия для краёв

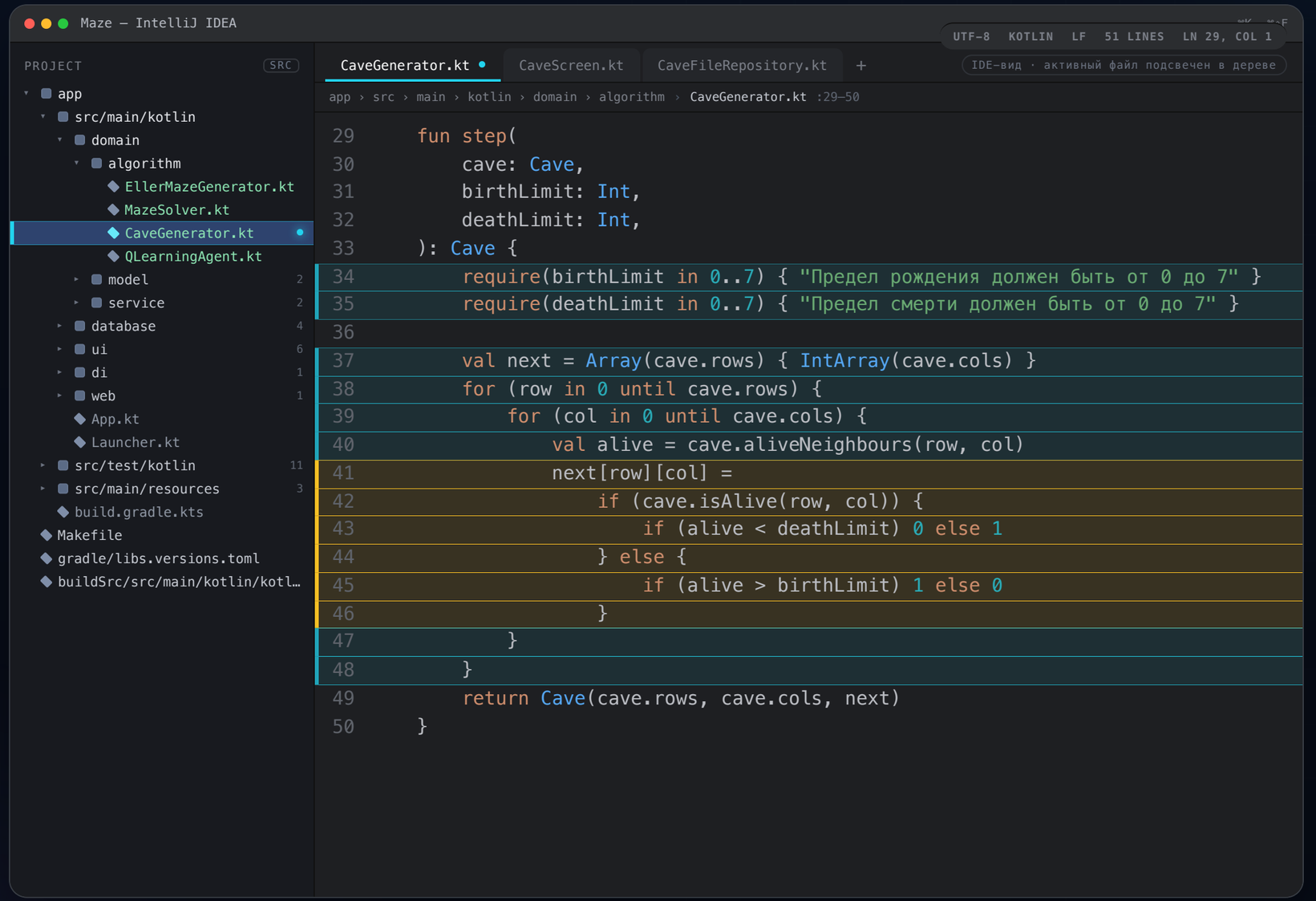
- У угла 5 живых соседей, у края 3, в центре 0 — поэтому края «затягивает» стенами.
- Если заменить 1 на 0, форма пещер станет другой; это осознанное требование ТЗ.

Почему это важно

- CaveGenerator.step считает по этой функции; тесты проверяют углы, края и центр.

Part 4: шаг автомата, UI и файлы

step() считает новое поле в отдельный массив. UI даёт «Шаг» и «Авто» с задержкой N мс; файл пещеры читается и сохраняется без второй матрицы.



Правила step

- Живая: если `alive < deathLimit` — умирает, иначе живёт.
- Мёртвая: если `alive > birthLimit` — рождается. Именно строгое > для рождения.
- Пределы 0..7 проверяются через require.

Двойная буферизация

- Результат пишется в `next`, а не в исходную пещеру.
- Иначе вторая половина поля считалась бы уже по изменённым данным.

UI и файлы

- `CaveScreen` : «Шаг» и «Авто»; `LaunchedEffect(running, delayMs)` делает delay(N) и step().
- Загрузка/сохранение: первая строка rows cols, дальше rows строк по cols чисел 0/1.
- 15 тестов CaveGeneratorTest, покрытие 100%.

Part 5: что такое агент и что значит «обучить»

Агент — это не нейросеть и не внешняя библиотека. Это объект с Q-таблицей: для каждой клетки четыре числа. Обучение — заполнить таблицу опытом.

S

Состояние

Клетка лабиринта, например (2,3). Всего rows × cols состояний.

```
s = Position(2, 3)
```

domain/model/Maze.kt

A

Действие

0 — вверх, 1 — вниз, 2 — влево, 3 — вправо. В стену нельзя.

```
a = 3 → canGo(s, вправо)
```

Maze.openNeighbors()

R

Награда

−1 за шаг, 0 за финиш. Максимум Q = минимум шагов.

```
r = −1 · на финише r = 0
```

trainEpisode(): FINISH_REWARD

π

Политика

argmax Q в текущей клетке; при равенстве — случайный выбор.

```
a* = argmax Q(s, ·)
```

bestAction()



$$Q(s,a) \leftarrow Q(s,a) + \alpha \cdot (r + \gamma \cdot \max Q(s') - Q(s,a))$$

Агент хранит **Q-таблицу** и фиксированные **maze + end**. Он не хранит маршрут — маршрут извлекается жадным проходом по Q.

ЧТО ЗНАЧИТ «ОБУЧЕНИЕ» Это не запоминание одного маршрута, а оценка действий. После каждого эпизода Q-значения сдвигаются к фактической награде; поэтому агент умеет выходить из любой стартовой клетки, а не только из той, которую видел.

PART 5 · МЕХАНИКА

Part 5: Q-learning — формула и цикл

Q(s,a) — ожидаемая награда, если из состояния s пойти в a и дальше действовать оптимально. После каждого шага таблица подтягивается к цели Беллмана.

ОБНОВЛЕНИЕ

$$Q(s,a) \leftarrow Q(s,a) + \alpha \cdot [\, r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a) \,]$$

На финише max Q(s′) = 0: эпизод закончился, будущего нет.

Q-ТАБЛИЦА · КЛЕТКА → 4 ДЕЙСТВИЯ

S	↑	↓	←	→
(2,3)	**12.4**	8.1	9.7	10.2
(2,4)	**14.8**	11.2	13.0	9.4
(3,3)	7.2	**10.5**	6.8	8.9

Зелёное — max Q. Политика агента: идти в это действие.

α
0.1
скорость обучения

γ
0.9
учёт будущего

ε
0.5 → 0.05
исследование → использование

E
max(50k, N·150)
эпизоды; 50×50 → 375k

1 random start → **2** ε-greedy выбор → **3** шаг + награда → **4** обновление Q → **5** до финиша / maxSteps

Один шаг обучения s=(2,3), a=вправо, r=−1, max Q(s′)=12.4, Q(s,a)=8.1

`8.1 + 0.1·(−1 + 0.9·12.4 − 8.1) = 8.306`

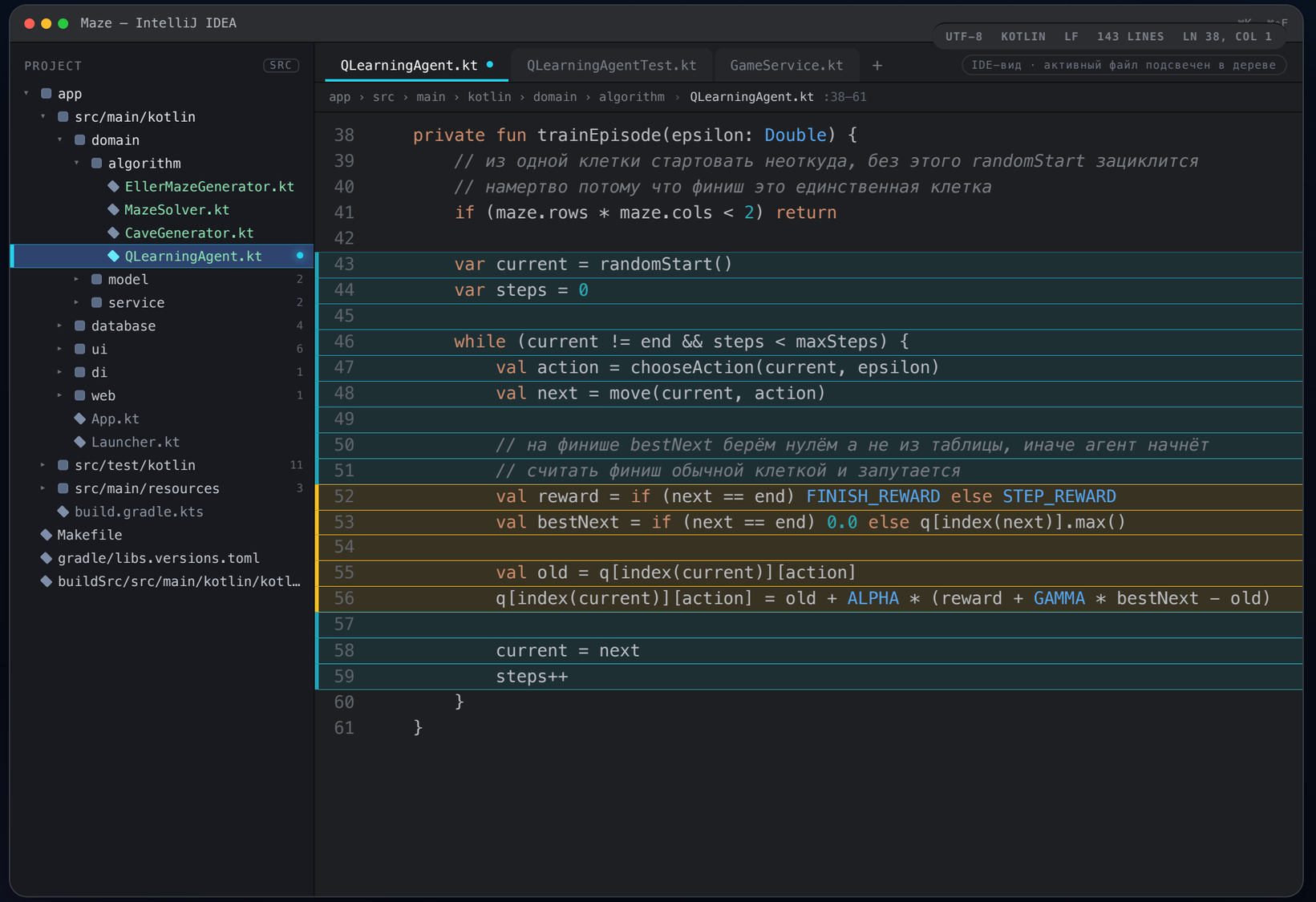
Почему −1 за шаг
Длинный путь суммарно хуже короткого — агент сам выбирает меньше шагов.

Почему γ = 0.9
Будущая награда чуть менее важна, но видна на десятки шагов вперёд.

Почему ε падает
Сначала исследовать все клетки, затем использовать найденный маршрут.

Part 5: код обучения агента

Один эпизод: случайный старт, шаги ε-greedy, награда, обновление Q. Ограничение maxSteps = клетки × 4 не даёт эпизоду зависнуть.



Разбор строк

- `randomStart()` — любая клетка, кроме финиша: так агент учится из любой точки.
- `chooseAction(current, epsilon)` — ε-greedy.
- `move` проверяет стену; если нельзя — остаёмся на месте и получаем −1.

Обновление

- `reward = if (next == end) 0 else -1`.
- `bestNext = if (next == end) 0 else q[next].max()` — терминальное состояние.
- Строка 56: `q[s][a] = old + α·(r + γ·bestNext − old)`.

Параметры и защита

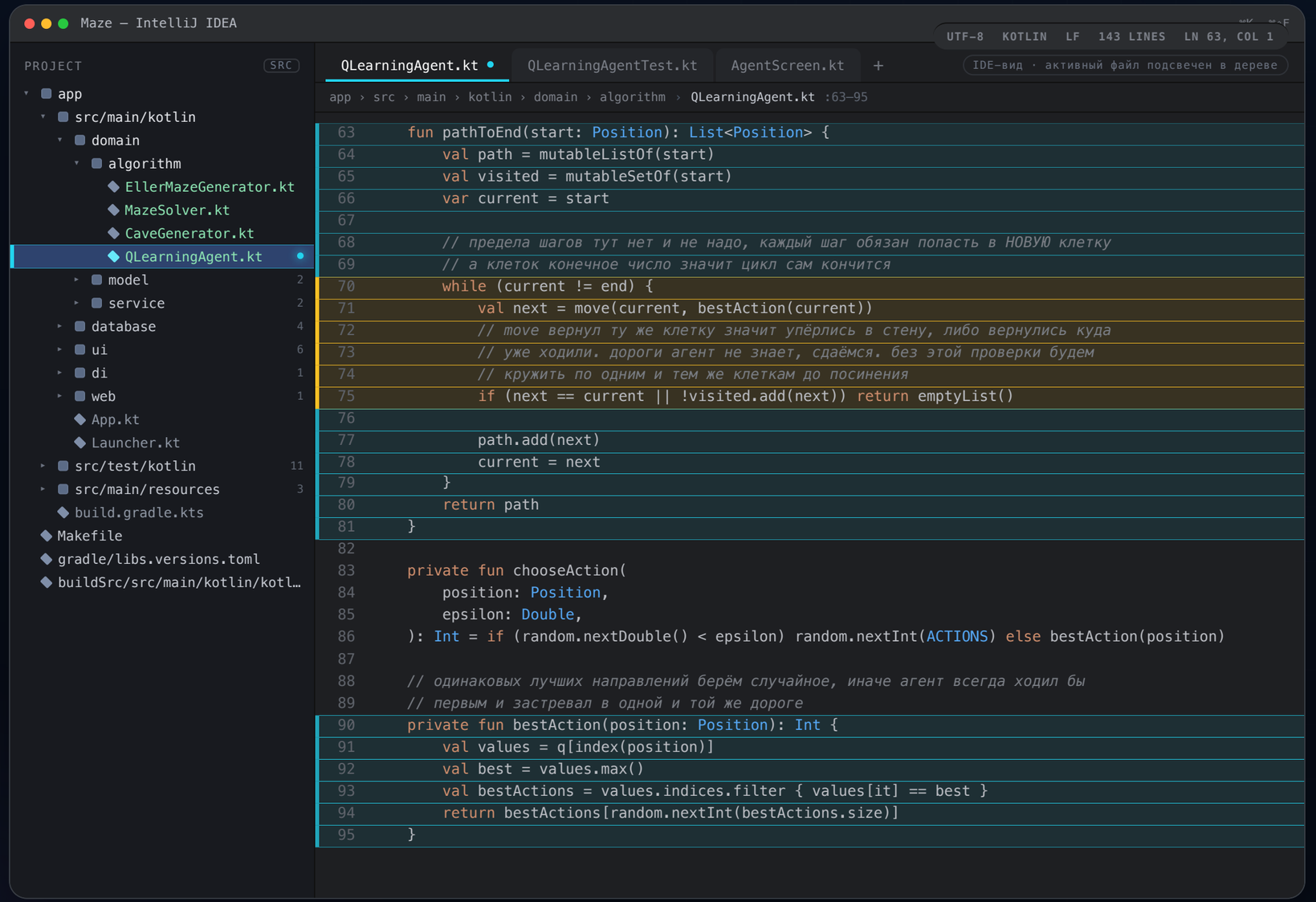
- Эпизоды: max(50 000, клетки × 150).
- Для 1×1 maze trainEpisode сразу выходит, иначе randomStart зациклится.
- `train(0)` и отрицательные значения отклоняются.

Почему maxSteps

- Защита от зацикливания: без лимита неверная политика кружит вечно.
- Лимит = клетки × 4 — с запасом больше любого простого пути.
- По достижении лимита эпизод обрывается, Q обновляется по последнему шагу.

Part 5: как из Q-таблицы получается маршрут

После обучения маршрут не ищется заново: агент просто идёт жадно по максимальному Q. visited защищает от циклов, стена — от невалидного шага.



pathToEnd

- Старт = финиш → [start].
- На каждом шаге bestAction(current); если move вернул ту же клетку (стена) или клетка уже посещена — emptyList.
- Проверка visited гарантирует завершение: клеток конечное число.

bestAction

- `values.max()` — лучшее Q; среди равных выбирается случайное.
- Случайность убирает постоянное предпочтение первого действия.

Когда пути нет

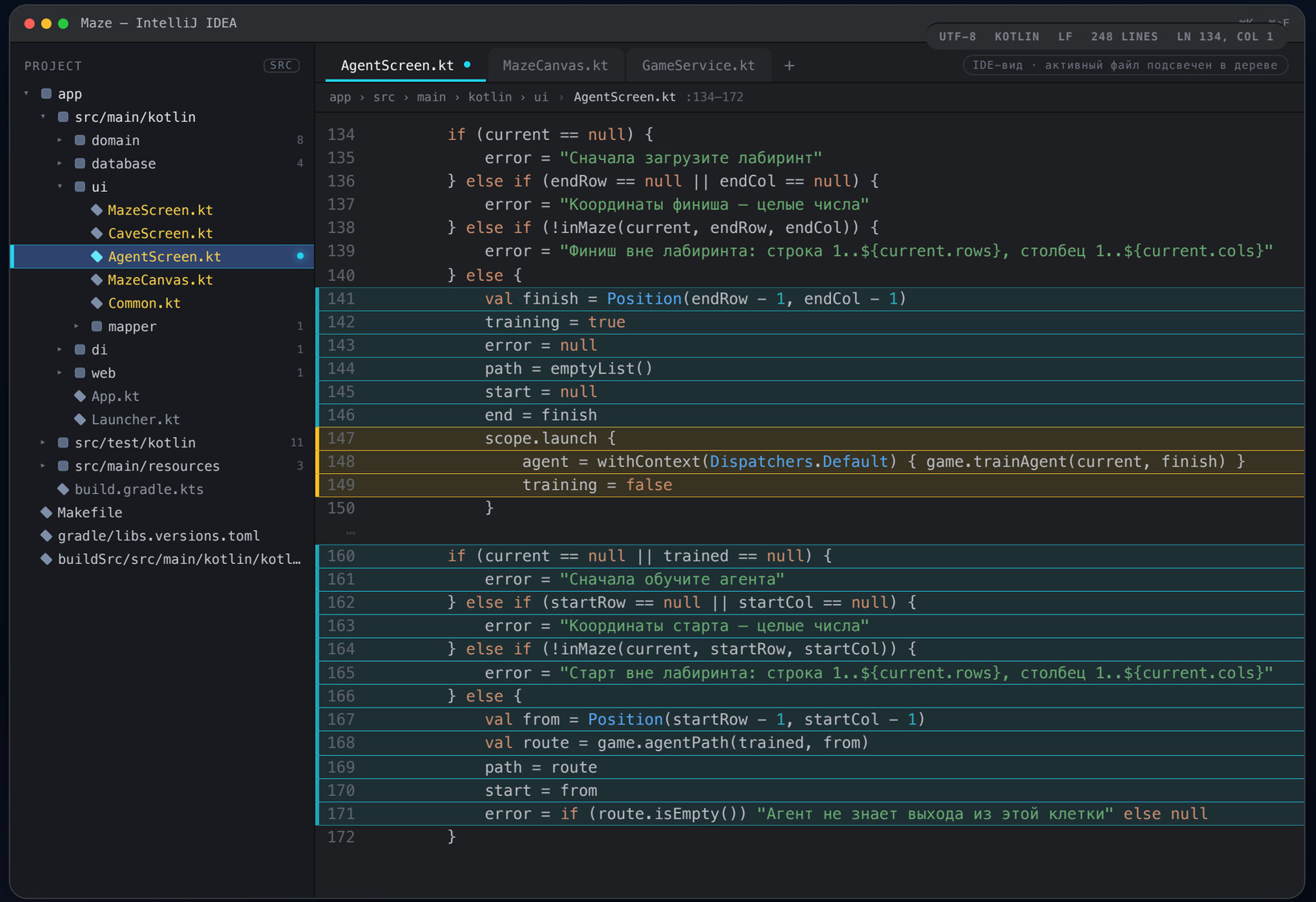
- Недостижимый финиш или обучение не сошлось — emptyList; UI пишет «Агент не знает выхода».
- Тесты покрывают недостижимый финиш, тупик, необученного агента и старт = финиш.

Граница ответственности

- pathToEnd ожидает start внутри поля; UI проверяет это до вызова.
- Для API стоит добавить require(start in field), но ТЗ этого не требует.
- Тот же путь рисует MazeCanvas — отрисовка не знает, кто его построил.

Part 5: взаимодействие с обученным агентом

Сценарий: загрузить файл → задать финиш → обучить в фоне → задать старт → показать маршрут. Новый лабиринт сбрасывает старого агента.



Обучение

- Финиш берётся из полей; проверка `inMaze`.
- Обучение уходит в `Dispatchers.Default` — окно не замирает.
- Пока идёт обучение, кнопки генерации/загрузки отключены.

Показ маршрута

- Старт задаётся полями или кликом; `game.agentPath(trained, from)`.
- Пустой результат → понятная ошибка «Агент не знает выхода из этой клетки».
- Маршрут рисуется тем же MazeCanvas, что и решение BFS.

Сброс

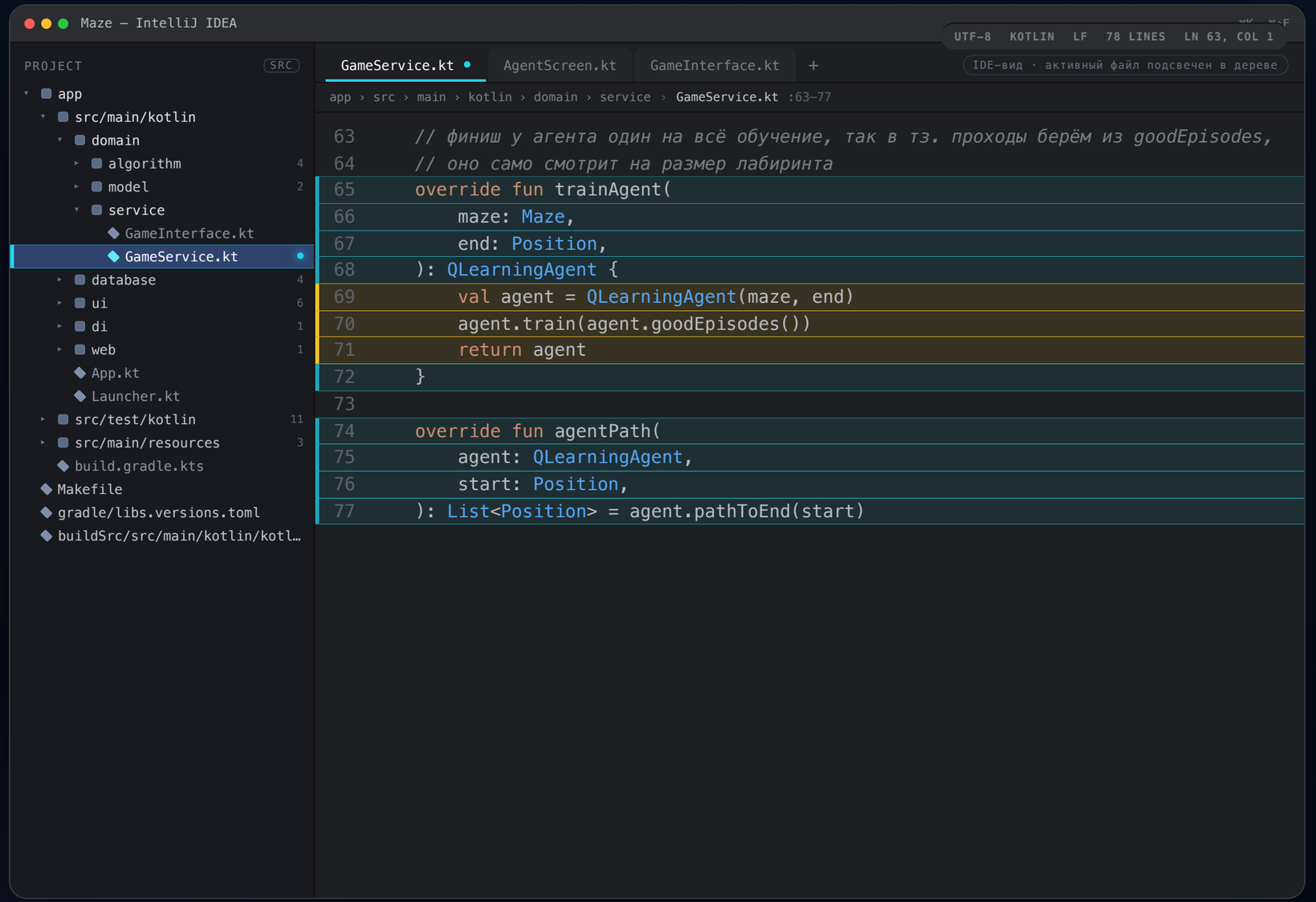
- `showMaze` ставит `agent = null`: агент обучен на другом лабиринте и не должен использоваться.
- При загрузке нового файла сбрасываются path, start и ошибки.

Почему в фоне

- scope.launch + withContext(Default): UI-поток остаётся отзывчивым.
- Пока training=true, кнопки обучения и маршрута заблокированы.
- Это Compose-состояние: при уходе с экрана корутина отменяется вместе с ним.

Part 5: изоляция данных и фиксация среды

Агент хранит собственные maze и end. GameService — единственная точка входа; UI не может подменить лабиринт у уже обученного агента.



Что зафиксировано

- `QLearningAgent(maze, end)` — оба параметра val конструктора.
- Q-таблица private; снаружи доступны только train() и pathToEnd().
- Новый maze/end = новый агент и новое обучение, старый не меняется.

Фасад

- `trainAgent` создаёт и обучает агента; `agentPath` только делегирует.
- UI зависит от GameInterface, а не от QLearningAgent напрямую.

Фон и сброс

- Обучение в `Dispatchers.Default` ; UI не блокируется.
- `AgentScreen.showMaze` обнуляет agent при новом лабиринте.
- Глобального состояния нет: AppContainer создаёт один GameService, но агент живёт в состоянии экрана.

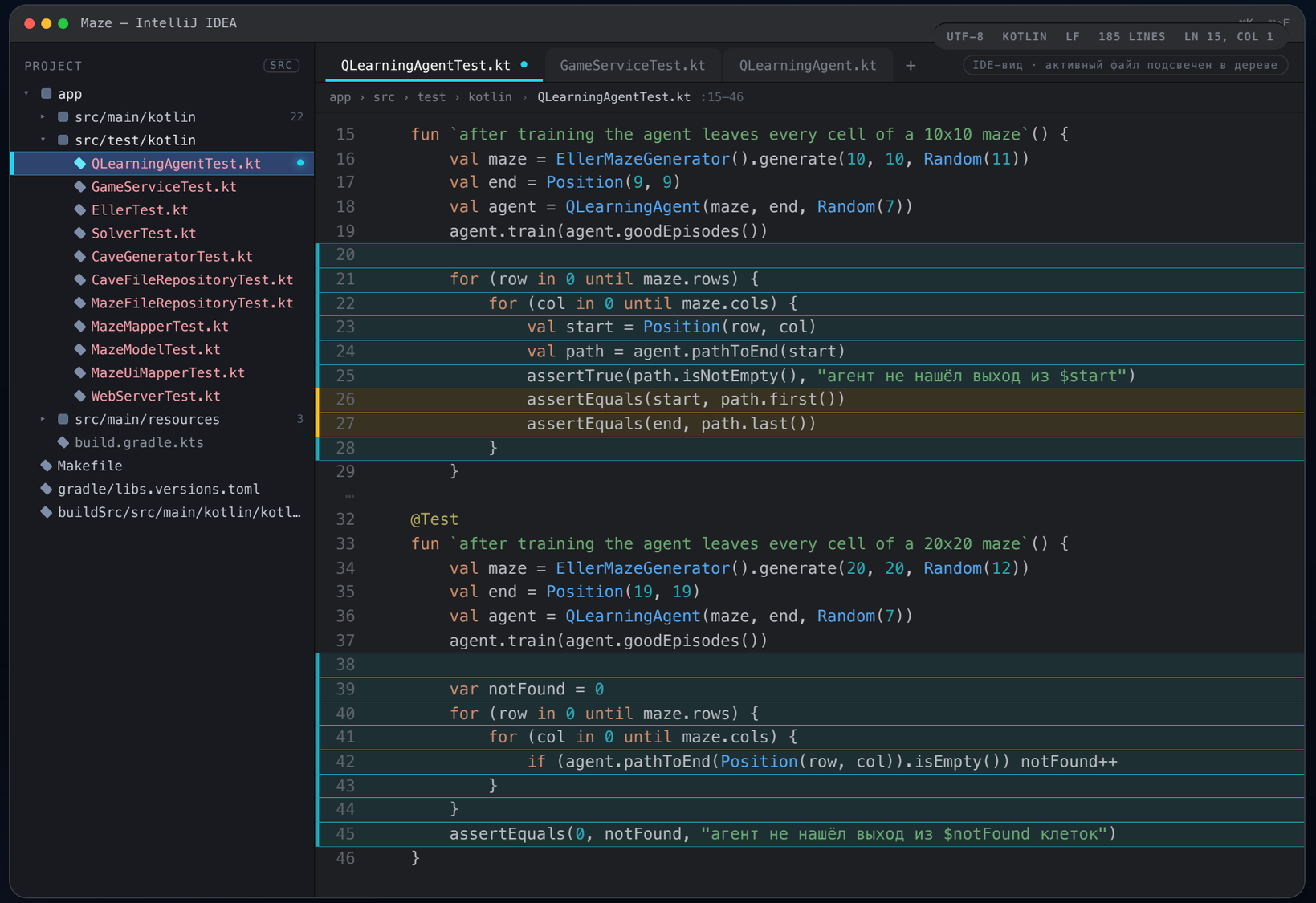
Что это гарантирует

- Нельзя обучить на одном лабиринте и построить путь на другом: maze/end — поля агента.
- trainAgent всегда создаёт нового агента; повторный маршрут обучение не запускает.
- GameInterface позволяет заменить реализацию, не меняя экраны.

GAMESERVICE.KT:63–77 · QLEARNINGAGENT.KT:13–18

Part 5: тесты и покрытие агента

13 тестов QLearningAgent + 3 интеграционных GameService. Покрытие модуля — 100% строк и ветвей; проверены все старты 10×10 и 20×20.



Что покрыто

- Все клетки 10×10 и 20×20: путь непустой, начинается в start, заканчивается в end.
- Маршрут не проходит через стены и не повторяет клетки.
- Старт = финиш, 1×1, недостижимый финиш, лабиринт без проходов, необученный агент.
- Детерминизм по seed, счётчик эпизодов, отказ при episodes ≤ 0.

Интеграция с BFS

- `GameServiceTest` сравнивает маршрут агента с `solveMaze` на идеальных лабиринтах.
- В идеальном лабиринте путь единственный, поэтому совпадение — строгая проверка кратчайшести.

Замер

- JaCoCo: QLearningAgent — 100% instruction/branch/line/method (54/54 строки, 50/50 ветвей).
- 20×20: все 400 стартов нашли выход. 10×10 на трёх лабиринтах: маршрут из каждой клетки совпал с BFS.

Почему 20×20, а не 50×50

- 50×50 — 375 000 эпизодов: такой unit-тест сильно замедлил бы CI.
- Свойства политики не зависят от размера; 50×50 проверяется запуском.
- 400 стартов 20×20 и 3×100 стартов 10×10 дают достаточную выборку.

PART 6 · WEB

Part 6: веб-интерфейс

Ktor отдаёт страницу и API; Vue рисует Canvas и ходит в API. Все алгоритмы остаются на Kotlin-сервере, в JS их нет.

ПОТОК ДАННЫХ



ЧЕГО В JS НЕТ

Генерации, BFS и Q-learning: страница только зовёт API и рисует ответ.

КАРТА API

МЕТОД	ПУТЬ	ЧТО ДЕЛАЕТ
GET	/	HTML-страница
GET	/vue.global.prod.js	Vue локально, без CDN
GET	/api/maze?rows&cols	Генерация; размеры clamp 1..50
POST	/api/maze	Разбор текста, ответ: maze + text
POST	/api/solve?...	BFS; 400 при ошибках

ОДИН ЗАПРОС «РЕШИТЬ»

1 fetch POST /api/solve · body: maze text + start/end
2 сервер: parse → validate → BFS → JSON path
3 200: path[] · 400: { error } → показ ошибки

JSON собирается вручную, без библиотеки. Координаты в API — с нуля; браузер переводит 1-based → 0-based.

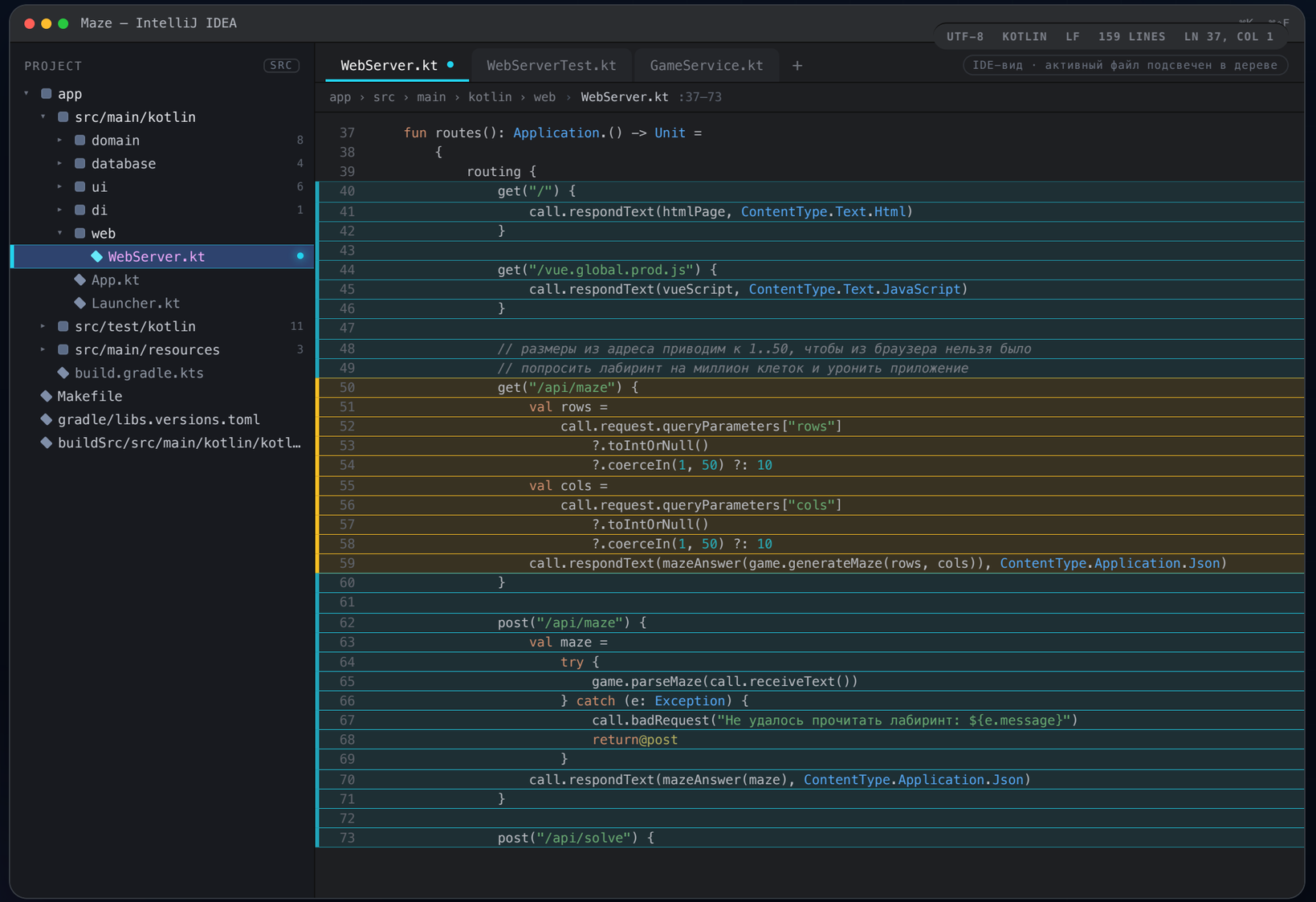
ПРОВЕРКА ИЗ ТЕРМИНАЛА

```
curl -X POST 'http://localhost:9090/api/solve?startRow=0&startCol=0&endRow=49&endCol=49' --data-binary @maze.txt
```

Объём Part 6: по ТЗ веб обязан покрывать Part 1–3 — здесь загрузка, генерация Eller и BFS. Q-learning-агент доступен в desktop-приложении.

Part 6: Ktor API — код

Роуты тонкие: разобрали вход, вызвали GameService, отдали JSON. Ошибки парсинга/координат — 400 и сообщение.



Почему так

- Страница только рисует; генерация и BFS — на сервере.
- Размеры из query coerce в 1..50, чтобы нельзя было попросить гигантский лабиринт.
- Ответ содержит и объект maze, и text в формате ТЗ — страница может скачать файл как есть.

Обработка ошибок

- `parseMaze` в try/catch → 400 с текстом.
- Для solve дополнительно проверяются целые координаты и попадание в поле.
- Все ответы — JSON; текст экранируется вручную.

Тесты

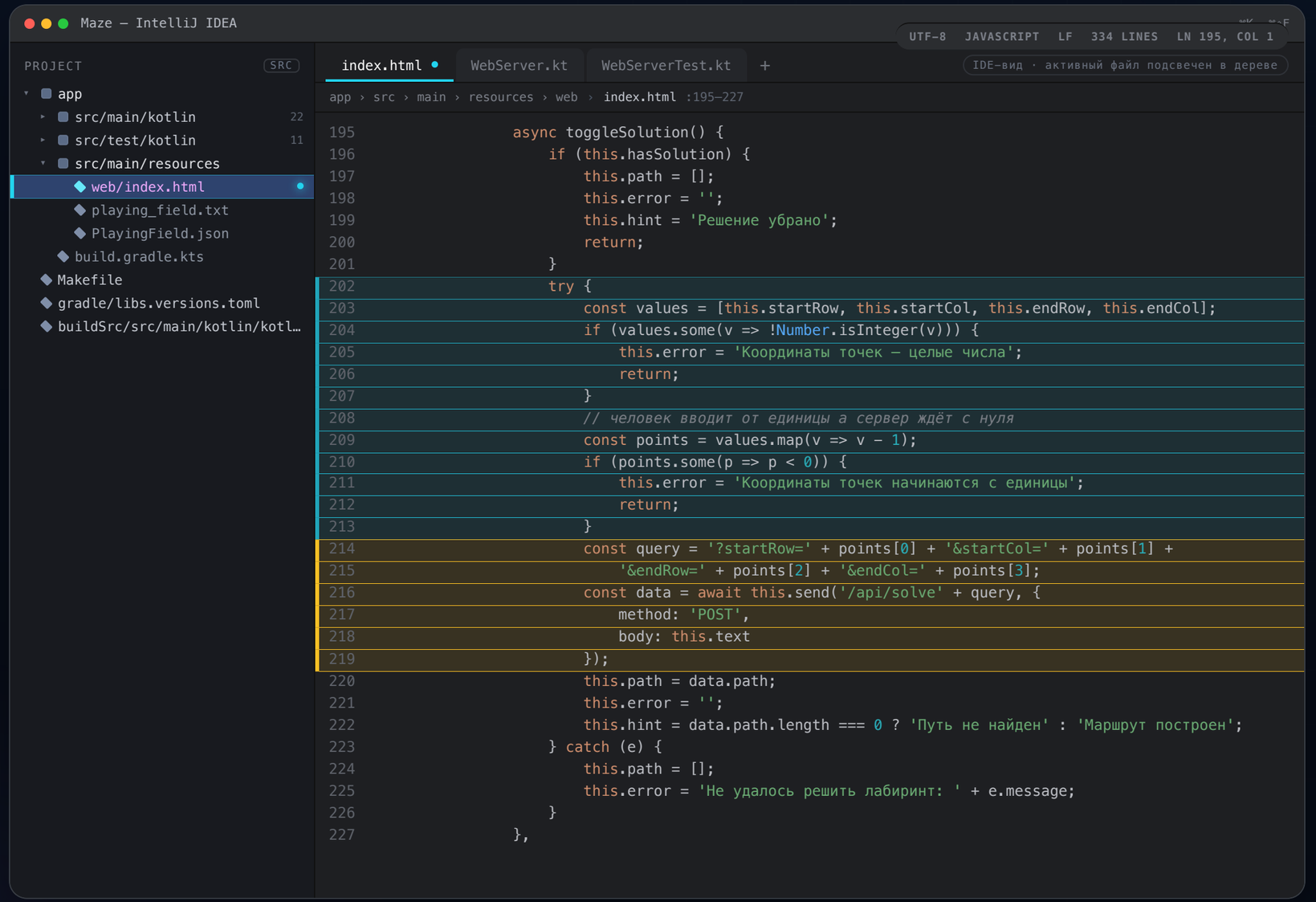
- WebServerTest: 11 тестов через Ktor test host.
- Генерация, загрузка, решение, старт = финиш, выход за границы, битые координаты, битый файл.

Почему JSON руками

- Нет kotlinx.serialization: меньше зависимостей и видно всё экранирование.
- textToJson экранирует кавычки, слэши и переводы строк.
- Ошибка всегда одного вида: 400 + {"error": "..."}

Part 6: Vue SPA — код

Клиент валидирует ввод, переводит координаты в 0-based, отправляет текст лабиринта и рисует ответ. Никакой логики лабиринта в JS.



Что делает Vue

- `send()` — fetch + разбор JSON + ошибка.
- `generate/load` — GET/POST и `showMaze`.
- `toggleSolution` — проверка целых, 1-based → 0-based, POST /api/solve.

Отрисовка

- `draw()`: canvas 500×500, WALL=2, стены #2E2E2E, маршрут #E53935.
- Точки старта/финиша рисуются даже без маршрута.
- Клик по canvas пересчитывает координаты с учётом масштаба.

Файлы

- Загрузка: file input → text → POST /api/maze.
- Сохранение: Blob из text, download maze.txt.
- Vue лежит локально: интернет для работы не нужен.

Как открыть

- Launcher выбирает свободный порт начиная с 9090 и открывает браузер.
- Ktor отдаёт index.html и vue.global.prod.js из resources.
- На защите: генерация → скачать файл → загрузить обратно → маршрут.

RESOURCES/WEB/INDEX.HTML:195–227 · DRAW(): 241–296

КАЧЕСТВО

Валидация: где что проверяется

Валидация распределена по слоям: файл, домен, UI, HTTP. Наружу всегда выходит понятное сообщение, а не stack trace.

СЛОЙ	ГДЕ	ЧТО ПРОВЕРЯЕТСЯ	РЕЗУЛЬТАТ
Файл лабиринта	`MazeFileRepository`	размер 1..50, ровно 1+2·rows строк, cols значений, только 0/1	IllegalArgumentException → UI error
Границы лабиринта	`MazeMapper`	правая и нижняя граница сплошные (1)	IllegalArgumentException
Файл пещеры	`CaveFileRepository`	размер 1..50, ровно 1+rows строк, cols значений, 0/1	IllegalArgumentException → UI error
Домен	`Eller / CaveGenerator / QLearningAgent`	размеры, chance 0..100, birth/death 0..7, episodes > 0	require → исключение
Desktop UI	`MazeScreen / CaveScreen / AgentScreen`	целые числа, диапазоны, координаты внутри поля	текст ошибки, приложение живо
Web server	`WebServer`	координаты — int, попадание в maze, тело парсится	400 + JSON error
Web client	`index.html`	целые, 1..50, координаты ≥ 1	сообщение в .error

ФАЙЛ → UI
`Число строк в файле не соответствует размеру лабиринта`

UI → ПОЛЬЗОВАТЕЛЬ
`Старт вне лабиринта: строка 1..50, столбец 1..50`

HTTP → 400
`{"error":"Координаты точек должны быть целыми числами"}`

Отдельно: **agentPath** сам не проверяет выход за границы — это делает UI. Для API-устойчивости можно добавить проверку в pathToEnd (не требование ТЗ).

ПРОВЕРКА

Тесты и качество

81 тест в 11 классах, все проходят. Алгоритмы покрыты на 100%; UI проверяется вручную, потому что это Compose-экран.

ПОКРЫТИЕ ПО JACOCO · СТРОКИ / ВЕТВИ



UI-слой (Compose) не входит в unit-покрытие: это декларативные экраны, их проверяем запуском приложения.

QLearningAgentTest	13
CaveGeneratorTest	15
WebServerTest	11
MazeFileRepositoryTest	8
CaveFileRepositoryTest	7
MazeModelTest	7
MazeUiMapperTest	6
SolverTest	6
GameServiceTest	3
MazeMapperTest	3
EllerTest	2

- `./gradlew test` — 81/81
- `./gradlew ktlintCheck` — чисто
- `./gradlew jacocoTestReport` — отчёт
- `make build` — сборка + тесты

450 / 450
инструкций QLearningAgent по JaCoCo

54 / 54 · 50 / 50
строк и ветвей QLearningAgent

400 / 400
стартов 20×20 дошли до финиша в тесте

ШПАРГАЛКА

Быстрая карта: что где искать

Если на защите спросят «где это?» — откройте эту таблицу. Пути указаны от src/maze.

ЧТО ИЩЕМ	ФАЙЛ	КЛЮЧЕВОЕ
Q-learning: обучение	`domain/algorithm/QLearningAgent.kt`	train / trainEpisode / Q-таблица
Маршрут агента	`domain/algorithm/QLearningAgent.kt`	pathToEnd / bestAction / move
Экран агента	`ui/AgentScreen.kt`	финиш → обучение → старт → маршрут
Фасад	`domain/service/GameService.kt`	trainAgent / agentPath / solveMaze
Генерация Эллера	`domain/algorithm/EllerMazeGenerator.kt`	sets / mergeSets / openings
BFS	`domain/algorithm/MazeSolver.kt`	queue / previous / buildPath
Пещера	`domain/algorithm/CaveGenerator.kt`	random / step
Файлы	`database/repository/*`	parse / save / toText
Отрисовка	`ui/MazeCanvas.kt`	500×500 / 2 px / path
Веб-API	`web/WebServer.kt`	GET/POST /api/maze, /api/solve
Веб-страница	`resources/web/index.html`	Vue 3 + canvas
DI / сборка	`di/AppContainer.kt · build.gradle.kts`	composition root / зависимости
Тесты	`app/src/test/kotlin`	11 классов, 81 тест

Q & A

Вопросы, которые могут задать

Короткие ответы, которые стоит держать в голове. Если нужен код — вернитесь к слайду с этим файлом.

1. Почему Q-learning, а не BFS?

ТЗ требует RL. Агент учится по наградам; BFS — только эталон в тестах и отдельный решатель Part 3.

2. Как гарантируется кратчайший путь?

Шаг −1, финиш 0 → максимизация Q = минимизация шагов. На 10×10 маршрут сверен с BFS; на 20×20 все 400 стартов нашли выход.

3. Почему старт случайный?

Чтобы покрыть все состояния: требование «из любой начальной точки». В тестах проверены все клетки.

4. Что если финиш недостижим?

Обучение не падает; pathToEnd возвращает пустой список, UI пишет ошибку. Есть отдельный тест.

5. Почему таблица, а не нейросеть?

Дискретное пространство до 50×50, 4 действия. Таблица проще, детерминированнее и не требует RL-библиотек.

6. Как фиксируется лабиринт и финиш?

Оба — val в конструкторе QLearningAgent. Новый лабиринт в UI сбрасывает агента; переобучение создаёт нового.

7. Где валидация?

Файл — repository/mapper; UI — экраны; HTTP — WebServer (400); домен — require. См. отдельный слайд.

8. Как связаны desktop и web?

Оба идут через GameInterface/GameService. Веб не дублирует алгоритмы: JS только рисует и вызывает API.

9. Почему Эллер даёт идеальный лабиринт?

N−1 проход, стены рвём только между разными множествами (нет петель), у каждого множества есть выход вниз (нет изоляции).

10. Как считаются клетки за границей пещеры?

Они живые: aliveNeighbours добавляет 1 за каждую клетку вне поля. Отсюда 5 соседей у угла и 3 у края.

ИТОГ

Что запомнить

Одна страница, которая остаётся в голове после защиты.

6
ЧАСТЕЙ ТЗ
лабиринт · генерация · решение · пещеры · агент · web

4
АЛГОРИТМА
Эллер · BFS · клеточный автомат · Q-learning

81
ТЕСТ
11 классов · все проходят · ktlint чисто

100%
ПОКРЫТИЕ АЛГОРИТМОВ
Eller · BFS · Cave · QLearningAgent

ФОРМУЛА, КОТОРУЮ СПРОСЯТ

$$Q(s,a) \leftarrow Q(s,a) + \alpha \cdot [\, r + \gamma \cdot \max Q(s') - Q(s,a) \,]$$

α = 0.1 · γ = 0.9 · ε: 0.5 → 0.05 · 375 000 эпизодов для 50×50

ЧТО СКАЗАТЬ ОДНОЙ ФРАЗОЙ

«Агент — это Q-таблица на 4 числа на клетку. Он учится эпизодами из случайных стартов, получает –1 за шаг и 0 за финиш, а маршрут потом строится жадно по максимальному Q.»

ЧТО ПОКАЗАТЬ
`make run` → вкладки «Лабиринт», «Пещера», «Агент»; затем Web.

ЧТО СКАЗАТЬ
6 частей, 4 алгоритма, 81 тест. Агент обучается сам — без RL-библиотек.

КУДА ОТКРЫТЬ
`QLearningAgent.kt` → `AgentScreen.kt` → `WebServer.kt`.

Открыть код: `src/maze/app/src/main/kotlin` · Тесты: `src/maze/app/src/test/kotlin` · Запуск: `make run`

Спасибо за внимание

Готов показать код, тесты и обучение агента.
Вопросы — по любому слайду.